



Cinvestav



COCYTEN  
CONSEJO DE CIENCIA Y TECNOLOGÍA  
DEL ESTADO DE NAYARIT

# LAB 4 BCD Decoder

**Student: Alonso Emmanuel López Macías**

Email's: [usuario\\_cocytén\\_32\\_x@cinvestav.mx](mailto:usuario_cocytén_32_x@cinvestav.mx)  
[alemlopezma@gmail.com](mailto:alemlopezma@gmail.com)

**Student: Cesar Eduardo Inda Cenicerós**

Email's: [usuario\\_cocytén\\_32\\_x@cinvestav.mx](mailto:usuario_cocytén_32_x@cinvestav.mx)  
[cesar.inda@iteso.mx](mailto:cesar.inda@iteso.mx)

**Full Professor: Dr. Martín González Pérez**

12/04/2025

**CINVESTAV**, Av Del Bosque 1145, El Bajío 45017, Zapopan Jalisco.

**COCYTEN**, Boulevard Luis Donaldo Colosio. Ciudad Industrial, 63173 Tepic Nayarit.

Technical Report

® CINVESTAV - COCYTEN

**Abstract:** *FPGA Terasic*

*Board – ALTERA Cyclone IV EP4CE115F29C7N*

*IDE – Quartus Prime 18.1*

*Boolean Algebra*

*Digital Gates & Logical Comparators*

*Git & GitHub Control Version*

**Keywords:** *ALUT, BCD, Decoder Combinational Logic, Anode, Segments, Display.*

## **Table of Contents**

|                                             |            |
|---------------------------------------------|------------|
| <b>Table of Contents.....</b>               | <b>ii</b>  |
| <b>Table of Figures .....</b>               | <b>iii</b> |
| <b>1.- INTRODUCTION .....</b>               | <b>0</b>   |
| <b>1.1.- Requirements.....</b>              | <b>0</b>   |
| <b>2.- FUNCTIONAL DESCRIPTION .....</b>     | <b>1</b>   |
| <b>2.1.- Boolean Algebra Equations.....</b> | <b>2</b>   |
| <b>2.2.- TrueTable.....</b>                 | <b>3</b>   |
| <b>2.3.- Results .....</b>                  | <b>4</b>   |
| <b>2.4.- GitHub Repository .....</b>        | <b>5</b>   |
| <b>3.- CHALLENGUES/ISSUES.....</b>          | <b>5</b>   |
| <b>4.- CONCLUSIONS.....</b>                 | <b>6</b>   |

## Table of Figures

|                                                    |          |
|----------------------------------------------------|----------|
| <b>Figure: 1 Bcd_Fpga .....</b>                    | <b>0</b> |
| <b>Figure: 2 BCDecoder_2Displays_Circuit .....</b> | <b>1</b> |
| <b>Figure: 3 Brief_Segment-a .....</b>             | <b>2</b> |
| <b>Figure: 4 Decoder Block .....</b>               | <b>2</b> |
| <b>Figure: 5 Boolean Equation .....</b>            | <b>2</b> |
| <b>Figure: 6 TrueTable.....</b>                    | <b>3</b> |
| <b>Figure: 7 Test1.....</b>                        | <b>4</b> |
| <b>Figure: 8 Test2.....</b>                        | <b>4</b> |
| <b>Figure: 9 Test3.....</b>                        | <b>4</b> |
| <b>Figure: 10 Test4.....</b>                       | <b>4</b> |
| <b>Figure: 11 GitHub Repository .....</b>          | <b>5</b> |

## 1.- INTRODUCTION

For this lab, the objective was to create a combinational logic circuit by implementing a decoder functionality block, which would generate the logical combinational outputs for the different segments of the BCD to be added to the board. To achieve this, the functionality elements of the block were used—specifically, integrating the logic of the ALUT component in this decoder representation—which allowed us to carry out the various functions and combinations without having to manually build the entire circuit for this key element. We were asked to generate the combinations from 0 to 9 using only 4 inputs to represent the 10 possible numbers with the 4-bit input.

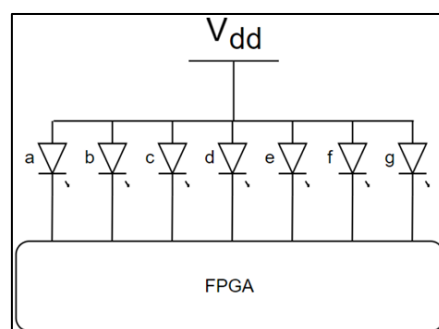
### 1.1.- Requirements

In this lab, you will design a circuit capable of displaying numbers 0 to 9 on a seven-segment display using the following methods:

- Using logic gates (Karnaugh maps are recommended).
- Using the decoder method.
- Using LUTs (Look-Up Tables). This will help analyze the advantages and disadvantages of each method.

The seven-segment displays on the DE2-115 FPGA board are common anode, meaning the LEDs are controlled using logic 0 (LOW) signals. Recommendations:

- Step 1 Create the truth table.
- Step 2 Derive the equations for each segment (LED) for logic gate implementation.
- Step 3 Obtain the sum of minterms equations for each segment to implement the decoder method.
- Step 4 Design the circuit using LUTs.
- Step 5 Implement the circuit on the FPGA.



*Figure: 1 Bcd\_Fpga*

## 2.- FUNCTIONAL DESCRIPTION

In this case, the project was developed using multiple OR gates with the purpose of allowing the inputs—when at least one condition is high (1) to be accepted by the gate. Additionally, before reaching each segment of the BCD, the input was inverted using a NOT gate. The decoder element was also used, which allowed for most of the circuit's functionality to be generated, and only the logical combinations up to the letter F were applied. This was implemented using two 7-segment BCD displays.

TEAM 14\_LAB4 BCD DECODER 16 COMBINATIONS  
ALONSO EMMANUEL LOPEZ MACIAS  
CESAR EDUARDO INDA CENICEROS

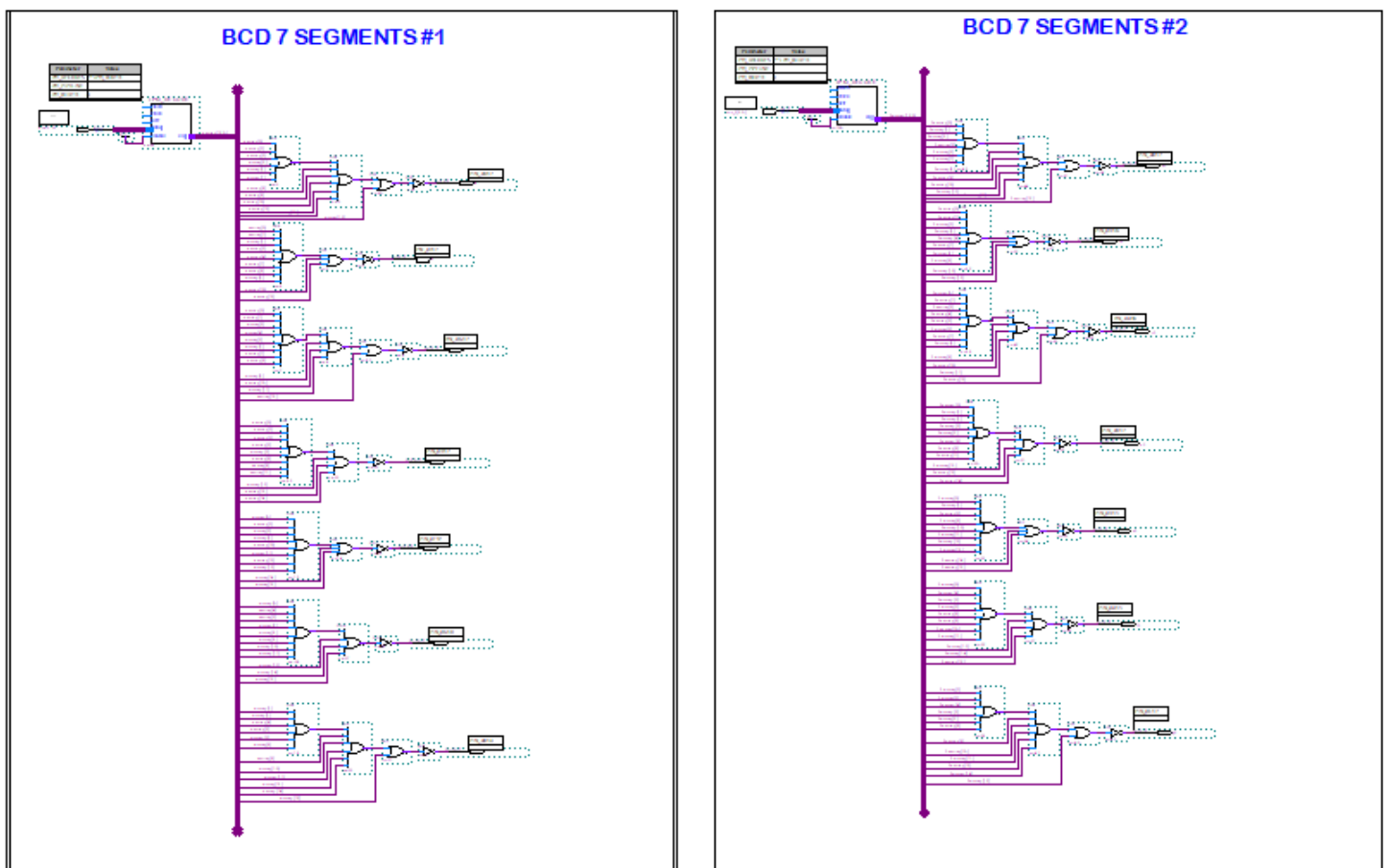


Figure: 2 BCDDecoder\_2Displays\_Circuit

For better visualization, for example, in the case of each BCD, it would appear as follows, considering the corresponding combination for each binary number according to the truth table.

For each one segment and for each one display.

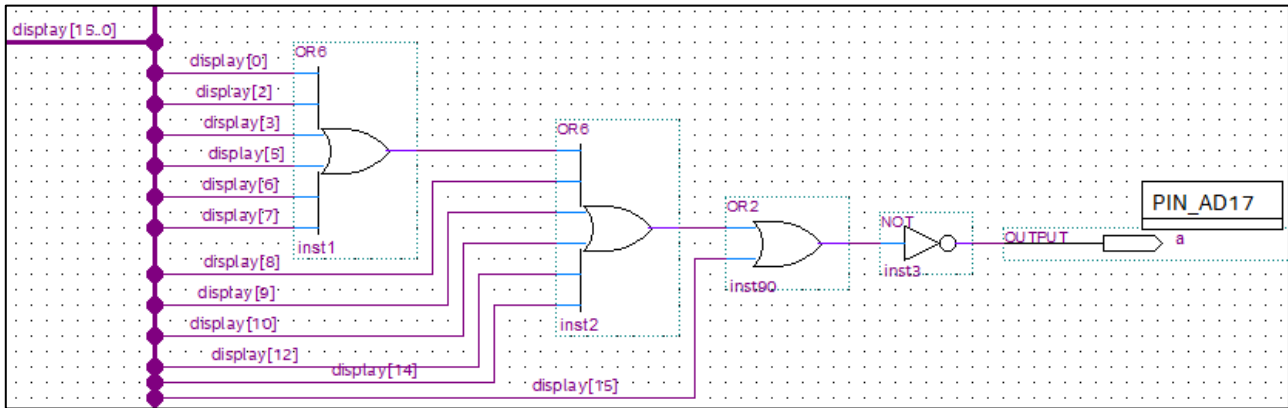


Figure: 3 Brief\_Segment-a

The block function for decode is the following:

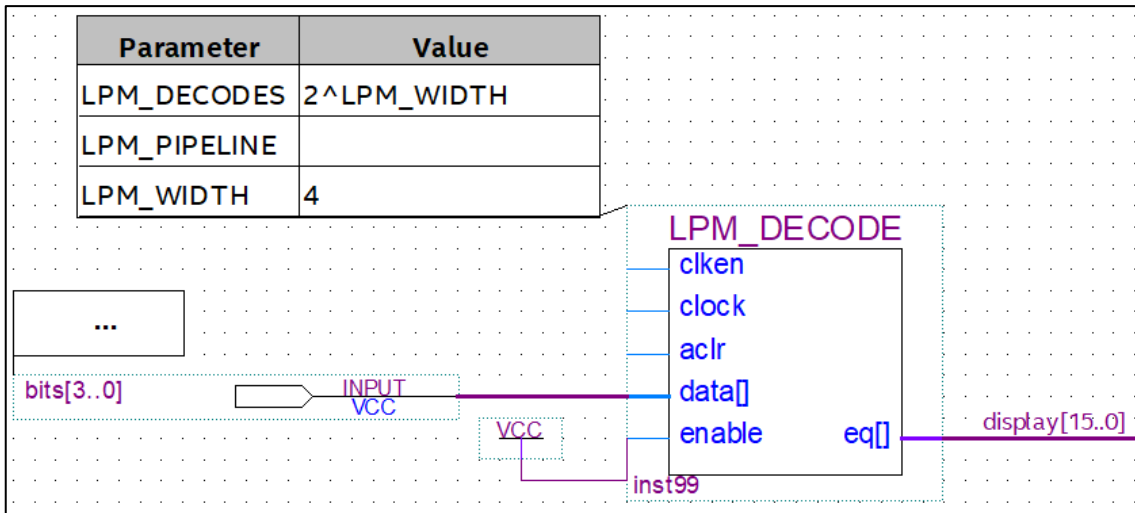


Figure: 4 Decoder Block

## 2.1.- Boolean Algebra Equations

For this implementation the equations are:

|   |                                                                                                            |
|---|------------------------------------------------------------------------------------------------------------|
| a | $\bar{A}BD + A\bar{B}\bar{C} + \bar{B}\bar{D} + \bar{A}C + A\bar{D} + BC$                                  |
| b | $\bar{A}(C \oplus D) + A(C \oplus D) + \bar{B}\bar{C} + \bar{B}\bar{D}$                                    |
| c | $\bar{B}\bar{C} + \bar{B}D + \bar{C}D + \bar{A}B + A\bar{B}$                                               |
| d | $\bar{B}\bar{C}\bar{D} + \bar{B}CD + \bar{A}\bar{C}\bar{D} + \bar{B}\bar{C}D + A\bar{C} + A\bar{B}\bar{D}$ |
| e | $\bar{B}\bar{D} + \bar{C}\bar{D} + AB + AC$                                                                |
| f | $\bar{A}\bar{B}\bar{C} + \bar{C}\bar{D} + \bar{B}\bar{D} + AC + A\bar{B}$                                  |
| g | $\bar{A}\bar{B}\bar{D} + \bar{B}\bar{C}\bar{D} + \bar{B}\bar{C} + AC + A\bar{B}$                           |

Figure: 5 Boolean Equation

## 2.2.- TrueTable

For this lab, we generated the truth table to implement the 16 combinations.

| NUMERO | BINARIO | <i>BCD (1 Display)</i> |          |          |          |          |          |          |
|--------|---------|------------------------|----------|----------|----------|----------|----------|----------|
|        |         | <i>a</i>               | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> | <i>f</i> | <i>g</i> |
| 0      | 0000    | 1                      | 1        | 1        | 1        | 1        | 1        | 0        |
| 1      | 0001    | 0                      | 1        | 1        | 0        | 0        | 0        | 0        |
| 2      | 0010    | 1                      | 1        | 0        | 1        | 1        | 0        | 1        |
| 3      | 0011    | 1                      | 1        | 1        | 1        | 0        | 0        | 1        |
| 4      | 0100    | 0                      | 1        | 1        | 0        | 0        | 1        | 1        |
| 5      | 0101    | 1                      | 0        | 1        | 1        | 0        | 1        | 1        |
| 6      | 0110    | 1                      | 0        | 1        | 1        | 1        | 1        | 1        |
| 7      | 0111    | 1                      | 1        | 1        | 0        | 0        | 0        | 0        |
| 8      | 1000    | 1                      | 1        | 1        | 1        | 1        | 1        | 1        |
| 9      | 1001    | 1                      | 1        | 1        | 1        | 0        | 1        | 1        |
| A      | 1010    | 1                      | 1        | 1        | 0        | 1        | 1        | 1        |
| B      | 1011    | 0                      | 0        | 1        | 1        | 1        | 1        | 1        |
| C      | 1100    | 1                      | 0        | 0        | 1        | 1        | 1        | 0        |
| D      | 1101    | 0                      | 1        | 1        | 1        | 1        | 0        | 1        |
| E      | 1110    | 1                      | 0        | 0        | 1        | 1        | 1        | 1        |
| F      | 1111    | 1                      | 0        | 0        | 0        | 1        | 1        | 1        |

Figure: 6 TrueTable

In this case, by applying the truth table for each display, it was possible to obtain the different parameters of the input bit combinations, allowing the behavior of each segment to be determined accordingly for each binary number entered. By manipulating both sets of switches, it is possible to obtain the highest number as FF, resulting in 16x16 combinations.

## 2.3.- Results

The complete tests are included in the video; for practical purposes in this report, a few demonstration values were provided for both displays to verify the correct functioning of the lab.

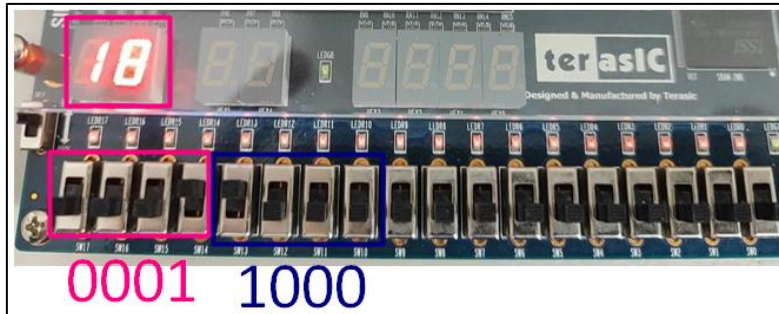


Figure: 7 Test1

BINARY1 = 0001  
BINARY2 = 1000

#1\_BCD = 1  
#2\_BCD = 8

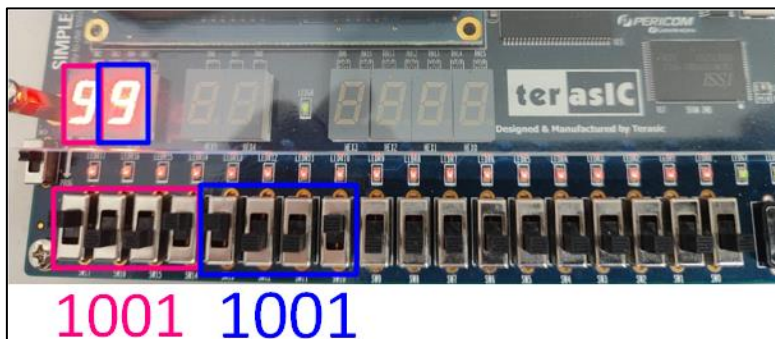


Figure: 8 Test2

BINARY1 = 1001  
BINARY2 = 1001

#1\_BCD = 9  
#2\_BCD = 9

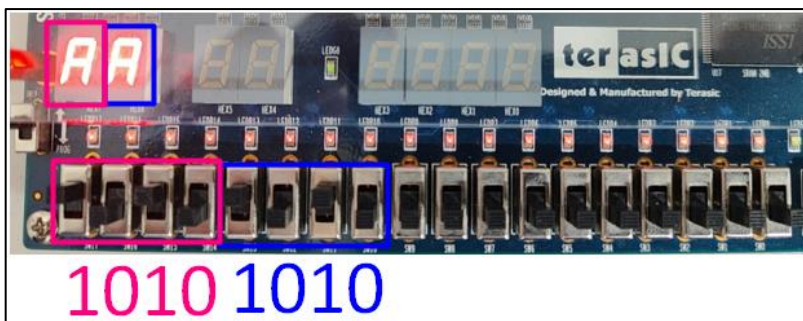


Figure: 9 Test3

BINARY1 = 1010  
BINARY2 = 1010

#1\_BCD = A  
#2\_BCD = A



Figure: 10 Test4

BINARY1 = 1111  
BINARY2 = 1111

#1\_BCD = F  
#2\_BCD = F



## 2.4.- GitHub Repository

A branch was created in the course repository corresponding to Lab4\_BCD-Decoder.

[https://github.com/CeicGitHub/Fundamentals\\_Of\\_Digital\\_Design\\_FPGA/tree/QuartusLab4\\_BCDDecoder](https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA/tree/QuartusLab4_BCDDecoder)

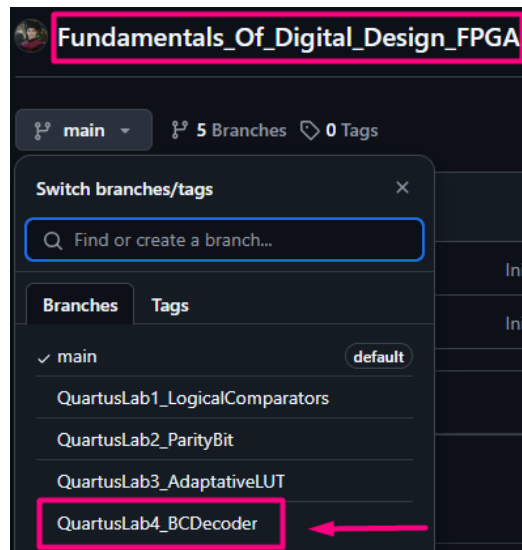


Figure: 11 GitHub Repository

## 3.- CHALLENGUES/ISSUES

### ***Alonso Emmanuel López Macías***

One of the main challenges of this practice was understanding how the decoder function block works. In general electronics, unlike in Quartus, it is easier to understand what goes in and what comes out. In Quartus, assigning parameters to the block—such as the enable signal or the eq[values]—required understanding which parameters to assign and what other aspects to consider during the design. I had difficulties when labeling the bus, as some instances and element names were duplicated or incorrect. This helped me become more familiar with the software and also understand that the logic of the gates would be defined for each individual segment.

### ***Cesar Eduardo Inda Cenicerros***

Personally, I found it somewhat challenging to understand how to manage the block, as I had to search online for some considerations—such as the fact that if the enable parameter is not activated with a power source in the program, it would show some errors related to VCC. Also, determining the logic of its operation based on the data buffer we assigned required understanding how those combinations were being displayed. Once we figured that out, it became easier to develop the remaining combinations.

## 4.- CONCLUSIONS

### ***Alonso Emmanuel López Macías***

This practice allowed me to learn a lot about electronics, as we are normally used to analyzing things from a physical perspective and don't always understand what's actually happening inside certain components. It helped me revisit my knowledge of electronic logic and programming, and to understand everything from the combinational circuit to gaining deeper insight into digital design within the Quartus software interface. I realized that these blocks, known as megafunctions, allow us to synthesize or reduce many processes that, in general electronics or what we are normally used to seeing, tend to be more extensive or complex.

### ***Cesar Eduardo Inda Cenicerós***

The practice gave me greater knowledge of the Quartus software, from understanding how these blocks function to, personally, finding it interesting how it's possible to use these elements and define many design parameters based on them. It allows us to understand a lot of what happens in electronics. I was also able to understand combinational logic from a different perspective, as grasping the concept of a communication bus and how it is possible to send a buffer with many values through it helps us determine much of what the program can do. It served as both a reminder and an improvement in my electronic programming logic.